

# Expert Viva Preparation Guide

## Dissolved Oxygen Prediction System

---

This document is a deeply humanized, expert-level preparation guide designed specifically for your Viva Voice. It breaks down technical concepts into easily understandable insights, perfect for answering tricky examiner questions.

### 1. Deep Dive: The Dashboard & Visual Results

**Q: What tools did you use to build the dashboard, and why only Streamlit?**

A: I used Streamlit entirely in Python. The reason I strictly chose Streamlit (instead of React, Angular, or Django) is because our core project is heavily Data Science and Machine Learning focused. Streamlit is natively integrated with Pandas, NumPy, and Keras. It allows us to deploy interactive machine learning predictive apps directly from our Python scripts without writing heavy HTML/CSS frontend logic. It provided the exact components we needed: dynamic sliders, metric cards, and instant Matplotlib chart rendering.

**Q: How does the threshold slider actually work under the hood?**

A: The threshold slider captures a user-defined float value (defaulting at 5.0 mg/L). Every time the model makes a DO prediction, the predicted value is compared against this slider's value in real-time. If the prediction drops below the threshold, a pandas DataFrame column 'alert' gets tagged as 'True'. Streamlit instantly reads this DataFrame and conditionally renders warning alerts and adjusts the metric cards.

**Q: How did you optimize the dashboard so it doesn't freeze or slow down?**

A: Streamlit has a quirk where it re-runs the entire python script every time a slider is moved. This means our heavy deep learning model would reload and predict data thousands of times! To fix this, I engineered a caching mechanism using the `@st.cache_data` decorator. Now, when a CSV is uploaded, the predictions are calculated precisely once and saved in local memory. Any changes to the UI slider instantly use the cached data, making the system run blazingly fast.

### 2. Explaining Project UML Diagrams

**Q: How would you explain your Use Case Diagram?**

A: The primary 'Actor' in our Use Case Diagram is the Aquaculturist (the farmer). Their main use cases are uploading sensor data, monitoring trends, and receiving alerts. Under the hood, these actions 'include' (via `<<includes>>` relationships) system sub-processes like

Data Preprocessing and Feature Selection, which happen automatically without the farmer's intervention.

**Q: What does the Activity Diagram represent in your project?**

A: The Activity diagram tracks the real-world flow of operations. It starts with raw Data Collection -> passes to Preprocessing (handling nulls/scaling) -> routes to LightGBM for Top-Feature extraction -> enters the BiSRU-Attention neural network -> and finally hits a decision node: 'Is DO < Threshold?'. If yes, it branches to 'Trigger Alert'; if no, it loops back to normal monitoring.

**Q: What is the architectural flow of your system?**

A: Our System Architecture is a strict sequential pipeline: Sensor Data -> Cleaning -> Feature Extraction (LightGBM) -> Sequence Modeling (BiSRU-Attention) -> Output Dashboard (Streamlit). Each layer acts as an isolated pipeline stage, allowing us to easily swap out models in the future without breaking the UI.

### 3. Core Machine Learning & Deep Learning Concepts

**Q: What exactly is Data Preprocessing, and why did you focus heavily on it?**

A: Data gathered in the real world is inherently 'dirty'—it has missing slots, negative erroneous values, or huge variations in scales. If we feed this raw data into an ML model, it learns 'garbage in, garbage out'. Preprocessing is cleaning it. Specifically, we used column medians to fill missing values (so we don't skew the distribution) and dropped negative rows.

**Q: What is Min-Max Normalization and why is it crucial?**

A: Min-Max Normalization mathematically squashes all features into a range between 0 and 1. Here is the logic: Temperature might be 30, but Ammonia might be 0.02. To a neural network, 30 is heavily prioritized over 0.02 just because it's a larger number! Normalizing guarantees the network treats every parameter equally, speeding up the 'Gradient Descent' convergence so the network learns faster and without bias.

**Q: What is the mathematical difference between Machine Learning and Deep Learning in your project context?**

A: Machine Learning (like our LightGBM) relies on human-crafted or structured statistical rules (like Decision Trees) to map inputs to outputs. It's fast but struggles with complex chronological sequences. Deep Learning (our BiSRU network) uses interconnected nodes that mimic the human brain (Neural Networks). It autonomously learns hidden temporal relationships over time sequences (the past 24 hours of water quality), which traditional ML cannot do efficiently.

**Q: Why Deep Learning for Time Series?**

A: Because traditional ML models treat every row of data as independent. But in nature, the oxygen level at 2:00 PM is heavily dependent on the oxygen level at 1:00 PM. Recurrent

Neural Networks (like our BiSRU) contain 'memory'—they pass the output of the past state into the current state as input, explicitly retaining chronological history.

## 4. Evaluation Metrics & Mathematical Breakdown

Evaluators love asking about the math behind accuracy. Here is how our model was measured:

**Mean Squared Error (MSE):** Our model achieved 0.0008.

*Formula:  $MSE = (1/n) * \sum (Actual - Predicted)^2$*

Explanation: It measures the average of the squares of the errors. We square it for two reasons: a) ensuring negative errors don't cancel out positive errors, and b) heavily punishing large outliers. An MSE near 0 is outstanding.

**Mean Absolute Error (MAE):** Our model achieved 0.0199.

*Formula:  $MAE = (1/n) * \sum |Actual - Predicted|$*

Explanation: It calculates the absolute average distance between predictions and actuals. Unlike MSE, it treats all errors equally without exponentially punishing outliers. It tells us that our DO predictions are, on average, incredibly close (off by only 0.0199) to reality.

**Root Mean Squared Error (RMSE):** Our model achieved 0.0285.

*Formula:  $RMSE = \sqrt{[(1/n) * \sum (Actual - Predicted)^2]}$*

Explanation: It is simply the square root of the MSE. We use RMSE because MSE's units are squared (e.g.,  $(\text{mg/L})^2$ ), which doesn't make logical sense when reporting. RMSE brings the error metric back to the original unit base (mg/L).

**R-Squared ( $R^2$  / Coefficient of Determination):** Our model achieved 96.28% (0.9628).

*Formula:  $R^2 = 1 - (SSR / SST)$*

Explanation: Where SSR is the sum of squared residuals and SST is the total sum of squares. In human terms: It represents the percentage of variance in Dissolved Oxygen that our model completely explains. 96.28% is phenomenal and proves our selection of LightGBM features and Attention mechanism perfectly capture the volatility in the water.

## 5. The Secret Sauce (Pro-Level Viva Answers)

**Q: What exactly does the 'Attention Layer' do mathematically?**

A: Standard neural networks force all 24 past hours of data into one summarized vector block, which causes data loss. The Attention layer calculates a mathematical 'Score' (using a Tanh activation function) for every single historical hour. It then converts these scores to probabilities (Weights) using a Softmax function. It multiplies the data by these weights—essentially saying 'Ignore hour 3, but zoom in on hour 19 because the ammonia spiked'.

**Q: Did your system have any overfitting? How did you control it?**

A: We strictly mitigated overfitting using 'Early Stopping' and a 'Validation Split' (15%).

During training, the model's performance on unseen validation data was checked every epoch. If the model started memorizing the training data instead of generalizing (meaning training loss went down, but validation loss went up), the Early Stopping mechanism forcibly halted the training process at the 5-epoch patience limit, actively restoring the best mathematical weights.

\*\*\* Best of luck with your Viva Voice! You've deeply understood and implemented highly advanced data science principles. Defend with confidence! \*\*\*